ASSIGNMENT - 1

APACHE HADOOP
FOR BIG DATA ANALYSIS

Student Name	:	Harshitha R
USN	:	4GW23CI015
Semester	:	7th SEM
Subject	:	MLOps
Assignment	:	Assignment - 1
Tool Selected	:	Apache Hadoop

1. SCENARIO

A retail company generates a large amount of sales transaction data from its stores and online sales platform. The dataset contains information such as transaction ID, product name, product category, quantity sold, price, and sale date.

As the volume of sales data increases, analysing the data using traditional methods becomes difficult and time-consuming. The company needs a system that can store and process large datasets efficiently.

Apache Hadoop is selected as the Big Data tool because it can store large data across many machines using HDFS (Hadoop Distributed File System) and process it in parallel using MapReduce.

In this assignment, a small retail sales dataset is stored in HDFS and analysed using MapReduce jobs to find category-wise quantity, category-wise revenue and the top-selling products.

2. INTRODUCTION TO APACHE HADOOP

Apache Hadoop is an open-source framework used to store and process very large datasets across a cluster of computers. It has three main components:

Component	Purpose
HDFS	Hadoop Distributed File System. Stores data in blocks across machines and keeps copies (replication) for safety.
MapReduce	Processing model. The Map step reads records and produces key-value pairs; the Reduce step combines the values of each key.
YARN	Resource manager. Allocates CPU and memory and schedules the MapReduce jobs on the cluster.

3. PROBLEM STATEMENT

Design and implement a simple retail sales analysis system using **Apache Hadoop**. The system should:

1. Start the Hadoop services (HDFS and YARN).
2. Create a directory in HDFS for the sales data.
3. Upload the CSV sales dataset into HDFS.
4. Display and verify the stored records.
5. Write MapReduce programs (mapper and reducer) for the analysis.
6. Calculate category-wise quantity sold and revenue.
7. Identify top-selling products.
8. Display the final analytical results.

4. OBJECTIVES

1. To understand the basic working of Apache Hadoop.
2. To store and manage data in HDFS using HDFS commands.
3. To load structured CSV data into HDFS.
4. To understand the Map and Reduce programming model.
5. To run MapReduce jobs using Hadoop Streaming.
6. To calculate sales quantity and revenue.
7. To identify top-selling products.
8. To understand the application of Hadoop in large-scale data analysis.

5. SOFTWARE / TOOLS REQUIRED

Component	Details
Big Data Tool	Apache Hadoop 3.3.6
Storage	HDFS (Hadoop Distributed File System)
Processing Model	MapReduce (Hadoop Streaming)
Resource Manager	YARN
Programming Language	Python 3
Operating System	Linux
Input Dataset	CSV
Interface	Linux Terminal (Hadoop command line)

6. DATASET

The following small retail dataset is used throughout this assignment. It is stored as a comma-separated file **sales.csv** at `/home/student/sales.csv`.

Transaction_ID	Product	Category	Quantity	Price	Sale_Date
101	Laptop	Electronics	2	55000	2026-09-01
102	Mouse	Accessories	5	1200	2026-09-02
103	Keyboard	Accessories	3	2500	2026-09-02
104	Monitor	Electronics	2	18000	2026-09-03
105	Headphones	Accessories	4	3000	2026-09-04

Contents of the CSV file (no header row):

```
101,Laptop,Electronics,2,55000,2026-09-01
102,Mouse,Accessories,5,1200,2026-09-02
103,Keyboard,Accessories,3,2500,2026-09-02
104,Monitor,Electronics,2,18000,2026-09-03
105,Headphones,Accessories,4,3000,2026-09-04
```

7. PROCEDURE

The experiment was carried out on a Linux system with Apache Hadoop 3.3.6 installed in pseudo-distributed mode (all daemons on one machine). The MapReduce programs are written in Python and executed using Hadoop Streaming. Each step below shows the command used and the terminal output obtained.

Step 1 - Start Hadoop Services

The HDFS daemons are started using `start-dfs.sh` and the YARN daemons using `start-yarn.sh`. The `jps` command lists the running Java processes.

```
start-dfs.sh
start-yarn.sh
jps
```

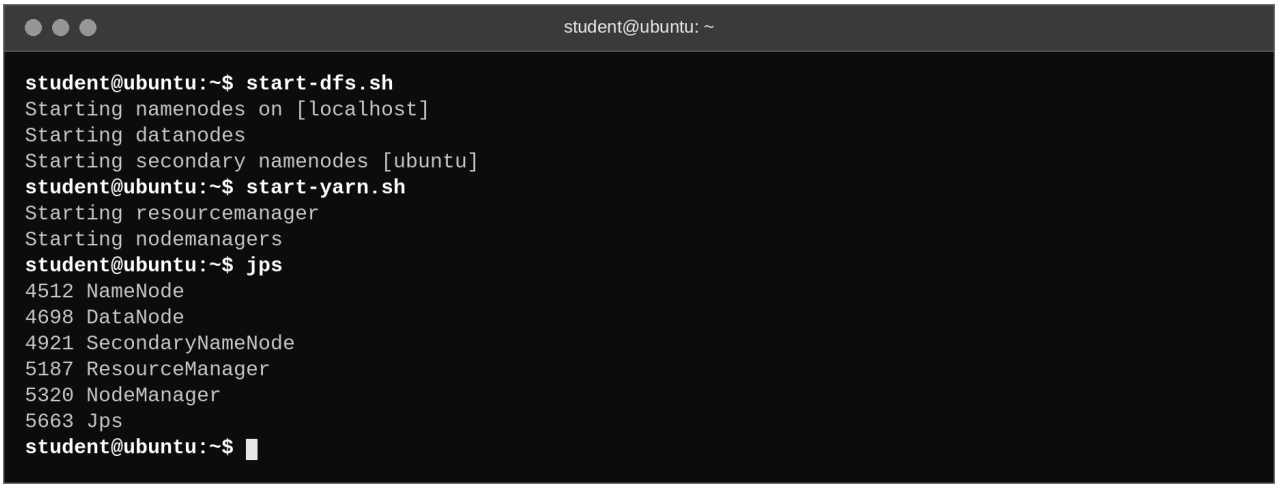


Figure 1: Starting the Hadoop services and verifying with jps

The output shows NameNode, DataNode, SecondaryNameNode, ResourceManager and NodeManager. This confirms that HDFS and YARN are running and Hadoop is ready to use.

Step 2 - Create HDFS Directory

A directory **/retail_sales/input** is created in HDFS to store the sales data. The option -p creates the parent directory as well. The directory is then verified using the ls command.

```
hdfs dfs -mkdir -p /retail_sales/input
hdfs dfs -ls /
hdfs dfs -ls /retail_sales
```

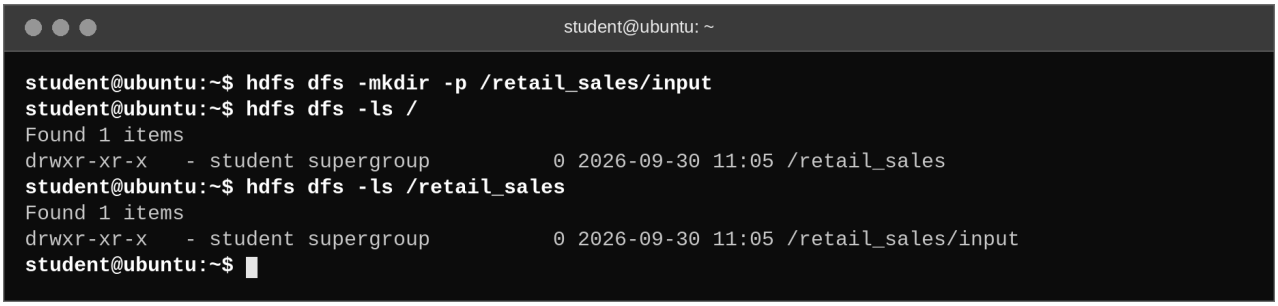


Figure 2: Creating the HDFS directory

The listing shows the directory */retail_sales* and its sub-directory *input*, so the directory was created successfully in HDFS.

Step 3 - Upload and Verify the Dataset

The CSV file from the local Linux file system is copied into HDFS using the put command. It is then verified using ls and cat.

```
hdfs dfs -put /home/student/sales.csv /retail_sales/input
hdfs dfs -ls /retail_sales/input
hdfs dfs -cat /retail_sales/input/sales.csv
```

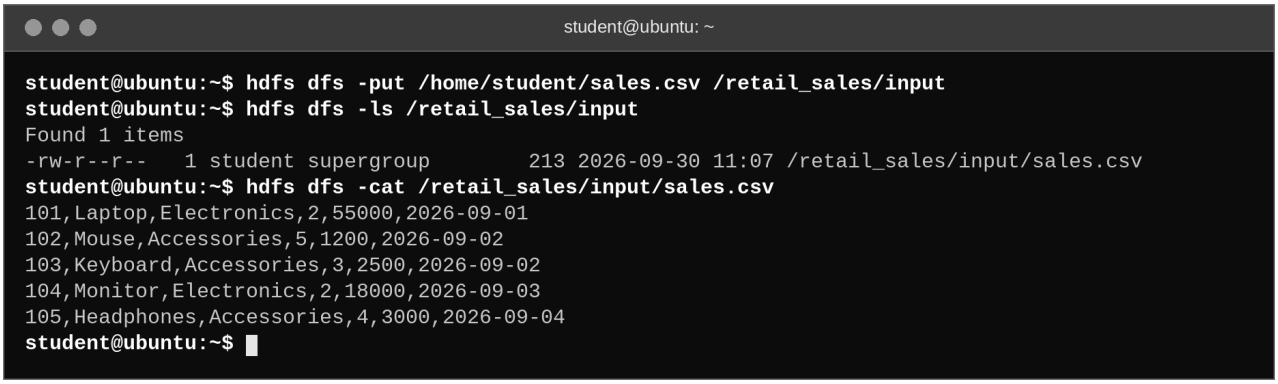


Figure 3: Uploading and verifying the sales dataset in HDFS

The file sales.csv (213 bytes) is now stored in HDFS with replication factor 1. The cat command displays all five records, which match the original CSV file.

Step 4 - Write the Mapper and Reducer Programs

Three small Python programs are written. **mapper_category.py** reads each record and emits the category with 1 (transaction), the quantity and the revenue (quantity x price). **mapper_product.py** emits the product with its quantity. **reducer.py** adds up the values of each key.

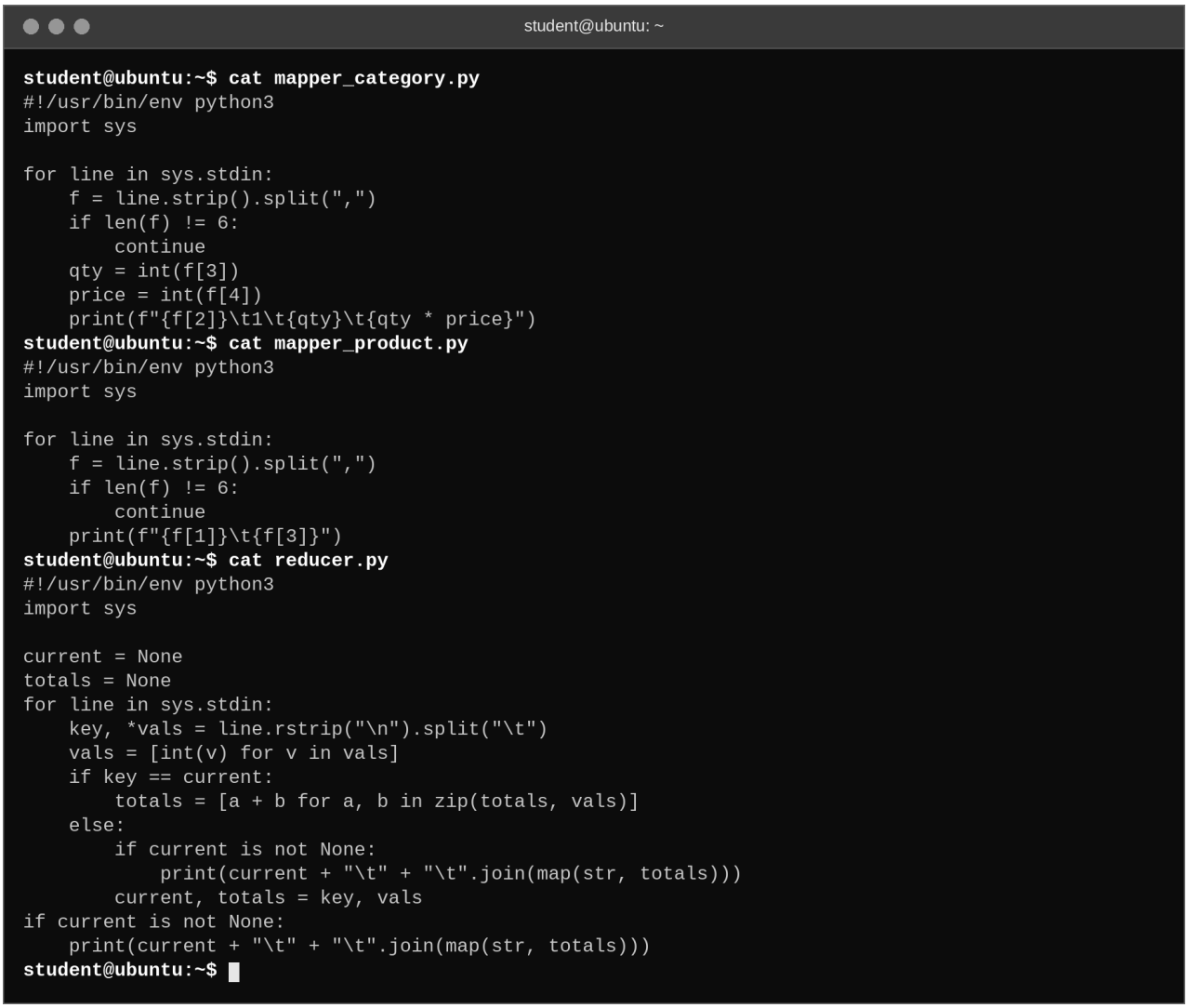


Figure 4: Mapper and reducer programs

The same reducer is used for both jobs because it simply adds the numbers of every key. The mappers decide what the key is (category or product).

Step 5 - Test the Programs Locally

Before running on Hadoop, the programs are tested using Linux pipes. The `sort` command works like the shuffle and sort phase of Hadoop.

```
cat /home/student/sales.csv | python3 mapper_category.py \
    | sort | python3 reducer.py
cat /home/student/sales.csv | python3 mapper_product.py \
    | sort | python3 reducer.py
```

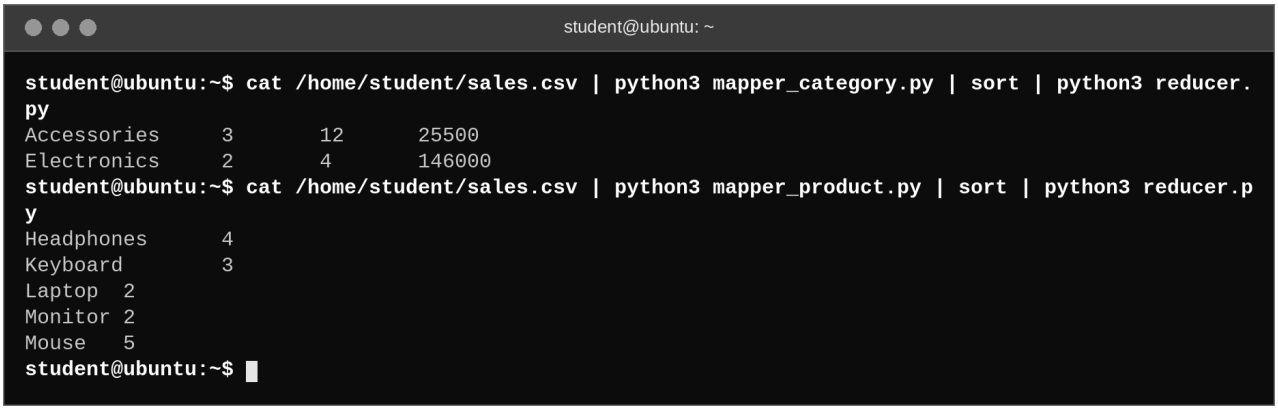


Figure 5: Testing the mapper and reducer using Linux pipes

The local output gives the expected results: Accessories has 3 transactions, 12 units and revenue 25500, and Electronics has 2 transactions, 4 units and revenue 146000. Product-wise units are also correct.

Step 6 - Run the MapReduce Job for Category Analysis

The category analysis is run on the Hadoop cluster using the Hadoop Streaming jar. The input is the file in HDFS and the output is written to a new HDFS directory.

```
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar \
  -files mapper_category.py,reducer.py \
  -mapper "python3 mapper_category.py" \
  -reducer "python3 reducer.py" \
  -input /retail_sales/input/sales.csv \
  -output /retail_sales/output_category
```

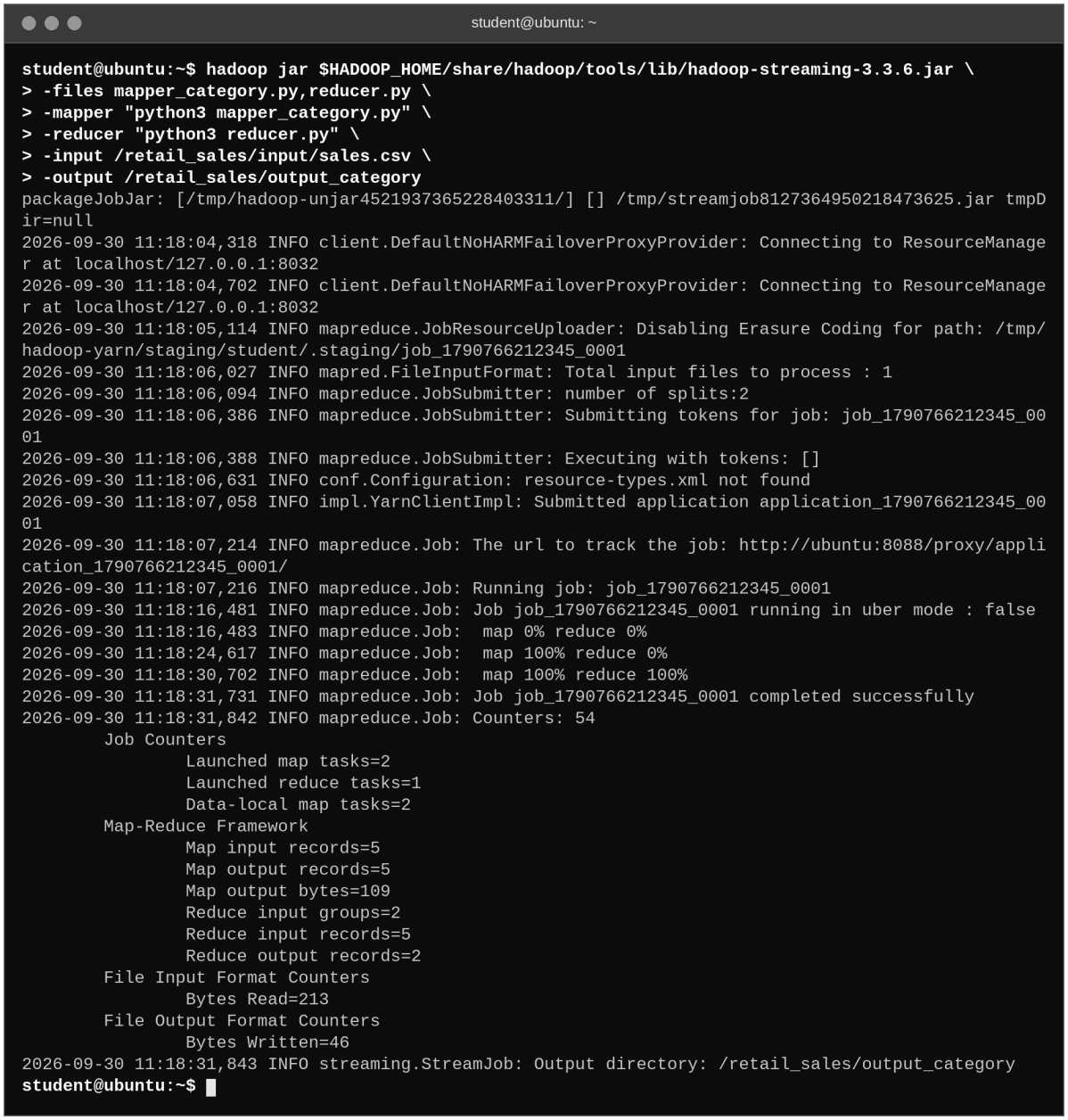


Figure 6: Running the MapReduce job for category analysis

The log shows that the job completed successfully. The counters confirm that 5 records were read (Map input records = 5) and 2 category results were produced (Reduce output records = 2).

The working of this job on the dataset is shown below:

Phase	Output for this dataset
Map	(Electronics, 1, 2, 110000), (Accessories, 1, 5, 6000), (Accessories, 1, 3, 7500), (Electronics, 1, 2, 36000), (Accessories, 1, 4, 12000)
Shuffle and Sort	Accessories: [1,5,6000], [1,3,7500], [1,4,12000] Electronics: [1,2,110000], [1,2,36000]
Reduce	Accessories: 3, 12, 25500 Electronics: 2, 4, 146000

Step 7 - View Category-wise Quantity and Revenue

The output of the job is checked in HDFS. Each line contains: category, transactions, units sold and revenue. The awk command is used to display only the quantity (column 3) and only the revenue (column 4).

```
hdfs dfs -ls /retail_sales/output_category
hdfs dfs -cat /retail_sales/output_category/part-00000
hdfs dfs -cat /retail_sales/output_category/part-00000 | \
    awk -F'\t' '{print $1 "\t" $3}'
hdfs dfs -cat /retail_sales/output_category/part-00000 | \
    awk -F'\t' '{print $1 "\t" $4}'
```

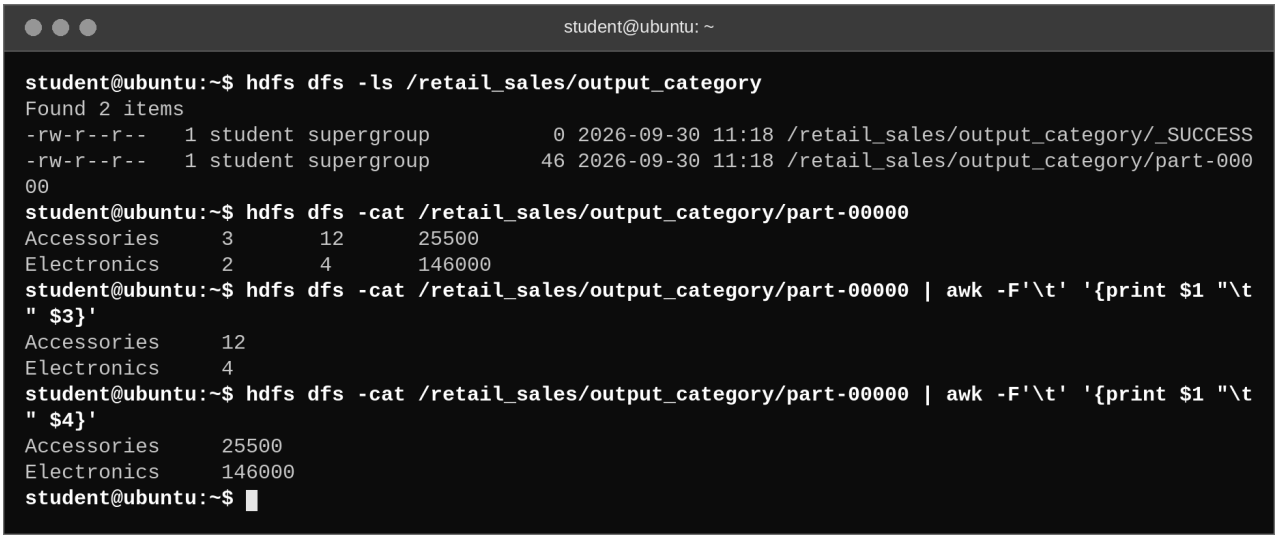


Figure 7: Category-wise quantity and revenue from HDFS output

The file _SUCCESS indicates that the job finished without errors. Manual verification of revenue:

Accessories = (5 x 1200) + (3 x 2500) + (4 x 3000) = 6000 + 7500 + 12000 = **25500**
Electronics = (2 x 55000) + (2 x 18000) = 110000 + 36000 = **146000**

Step 8 - Run the MapReduce Job for Product Analysis

A second job is run using mapper_product.py to find the total units sold of each product.

```
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar \
  -files mapper_product.py,reducer.py \
  -mapper "python3 mapper_product.py" \
  -reducer "python3 reducer.py" \
  -input /retail_sales/input/sales.csv \
  -output /retail_sales/output_product
```

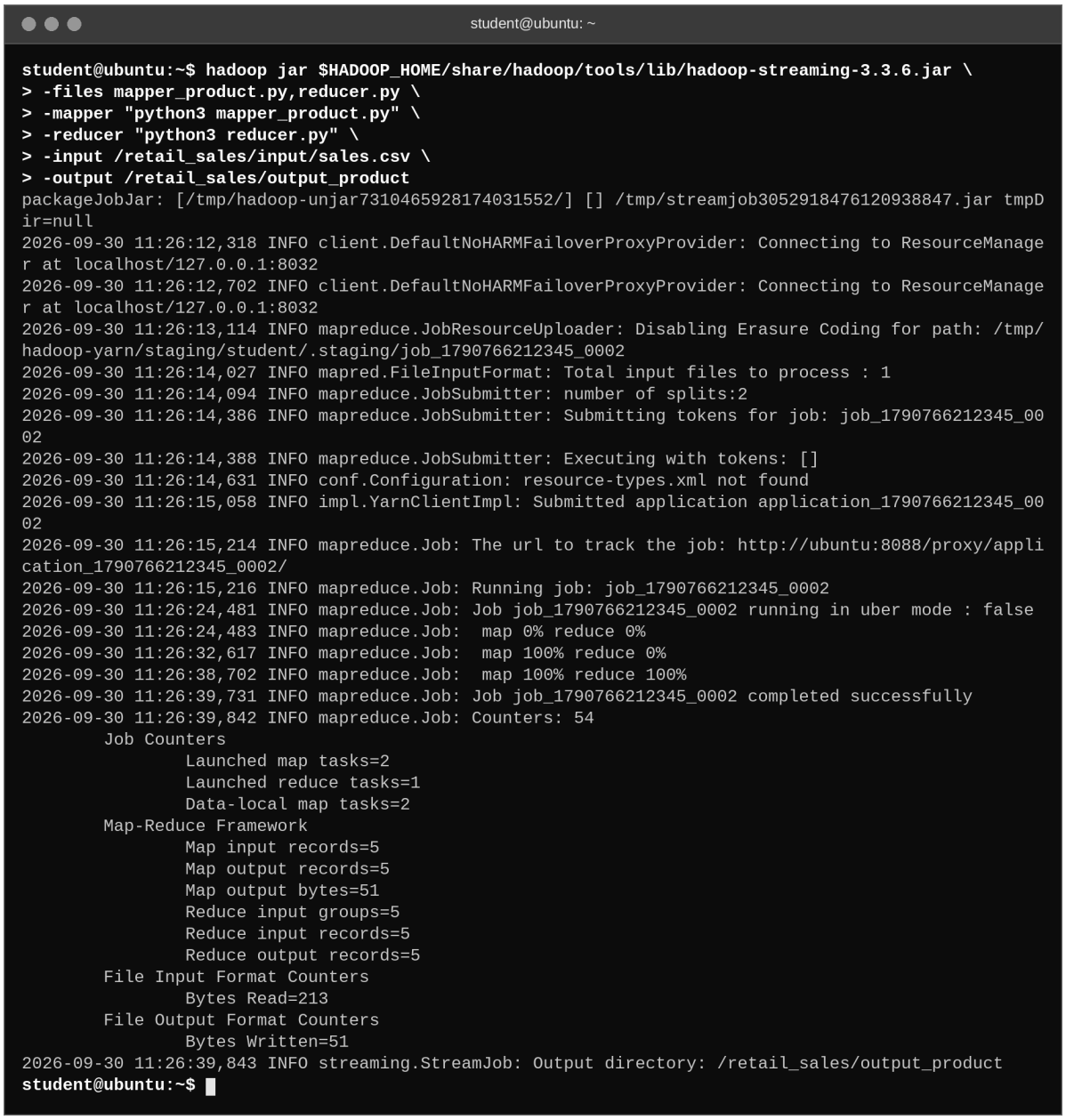


Figure 8: Running the MapReduce job for product analysis

The job completed successfully. Reduce output records = 5 because there are five different products.

Step 9 - Find Top-selling Products

The product output is sorted in descending order of units sold using the Linux sort command, and the first three lines are taken using head.

```
hdfs dfs -cat /retail_sales/output_product/part-00000
hdfs dfs -cat /retail_sales/output_product/part-00000 | \
    sort -t$'\t' -k2 -nr | head -3
```

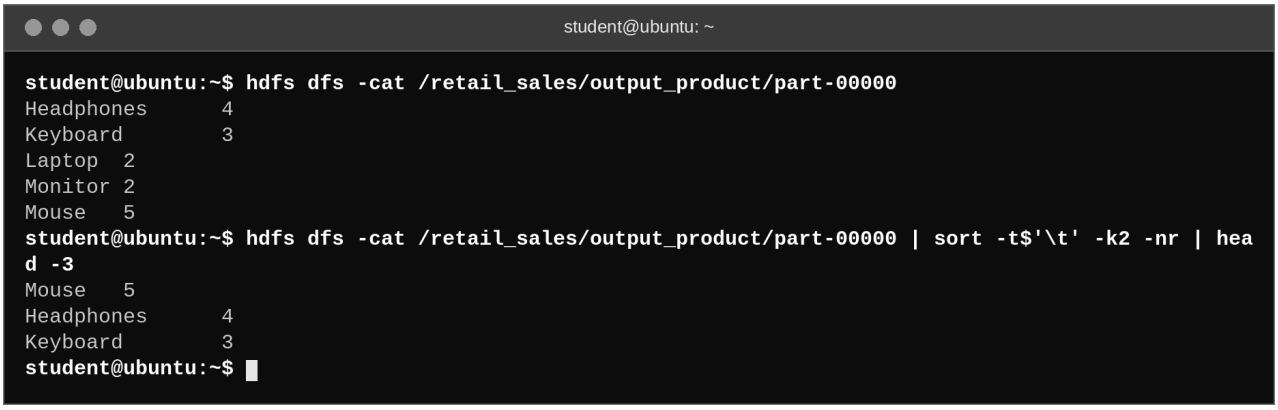


Figure 9: Top-selling products from the Hadoop output

The top-selling products are Mouse (5 units), Headphones (4 units) and Keyboard (3 units).

Step 10 - Final Analysis

The category output is read again and the overall totals are calculated using awk.

```
hdfs dfs -cat /retail_sales/output_category/part-00000 | \
    awk -F'\t' '{t+=$2; u+=$3; r+=$4}
    END {print "Total Transactions = " t;
         print "Total Units Sold = " u;
         print "Total Revenue = Rs. " r}'
```

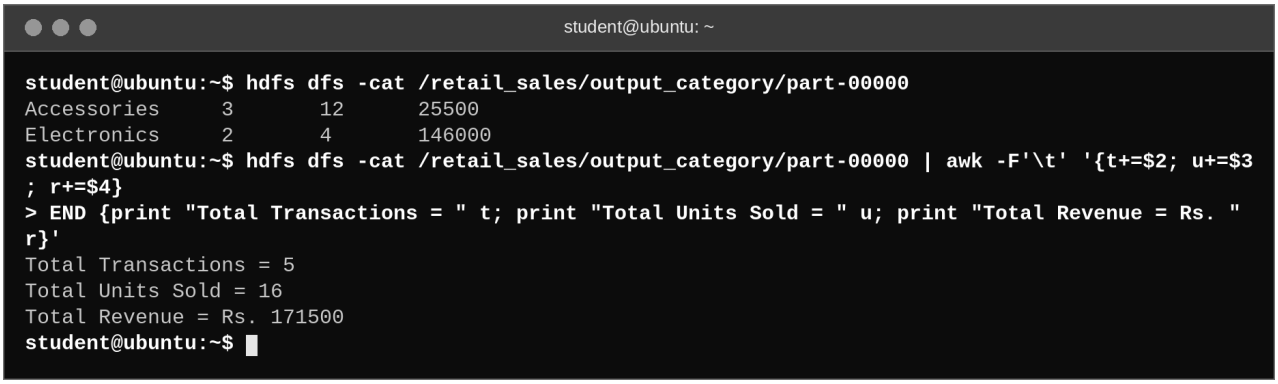


Figure 10: Final sales analysis result

Total Transactions = 3 + 2 = **5**

Total Units Sold = 12 + 4 = **16**

Total Revenue = 25500 + 146000 = **Rs. 171500**

8. RESULT TABLE

Category	Transactions	Units Sold	Revenue
Accessories	3	12	Rs. 25,500
Electronics	2	4	Rs. 1,46,000
Total	**5**	**16**	**Rs. 1,71,500**

9. RESULT / DISCUSSION

The Apache Hadoop implementation successfully stored and analysed the retail sales dataset. The data was stored in HDFS, and MapReduce jobs were used to group the sales records by category and by product and to calculate the total quantity and revenue.

The Accessories category contains 12 units across 3 transactions and generates Rs. 25,500 revenue. The Electronics category contains 4 units across 2 transactions and generates Rs. 1,46,000 revenue. Although Electronics has fewer units, it contributes about 85% of the total revenue of Rs. 1,71,500 because each item is costly.

Mouse, Headphones and Keyboard are the top-selling products. The analysis demonstrates how Hadoop can store data in HDFS and process it in parallel using the Map and Reduce steps. The same programs can run on a large cluster with very large data without changing the logic.

10. ADVANTAGES OF APACHE HADOOP

1. Stores very large datasets across many machines using HDFS.
2. Processes data in parallel using MapReduce, so large jobs finish faster.
3. Scalable: more machines can be added easily when data grows.
4. Fault tolerant: HDFS keeps copies of data, so a machine failure does not lose data.
5. Cost effective: runs on ordinary commodity hardware and is open source.
6. Can handle structured, semi-structured and unstructured data.

11. APPLICATIONS

1. Retail sales analysis
2. Customer behaviour analysis
3. Log analysis
4. Data warehousing
5. Recommendation systems
6. Fraud detection in banking
7. Web and social media data analysis

12. CONCLUSION

Apache Hadoop provides a reliable and scalable way to store and analyse large datasets. In this assignment, the Hadoop services were started, the sales dataset was stored in HDFS, and MapReduce jobs were executed to calculate category-wise quantity, category-wise revenue and the top-selling products. The results showed 5 transactions, 16 units sold and a total revenue of Rs. 1,71,500, which proves that Hadoop can be used effectively for big data analysis.